

Spatial Navigation using Grid Cells

```
In [24]: import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import convolve1d, convolve
from tqdm import tqdm
```

Exercise 0 — Getting Started: Ring Attractors in 1D

We begin with a 1D continuous ring attractor model. We simulate a field model integrated via the forward Euler method. The potential $s(x, t)$ of a neuron at position x evolves according to:

$$\tau \frac{ds(x, t)}{dt} = -s(x, t) + I_{rec}(x, t) + B(x, t)$$

where $B(x, t)$ is an external input to the neuron, and the recurrent input $I_{rec}(x, t)$ is defined as the convolution of each neuron's firing rate $r(x, t)$ with a synaptic weight kernel W that depends on the distance between two neurons:

$$I_{rec}(x, t) = \int W(x - x') r(x', t) dx'$$

The firing rate $r(x, t)$ of a neuron at position x is given by a simple ReLU activation on the potential:

$$r(x, t) = \max(0, s(x, t))$$

The recurrent connectivity kernel $W(x)$ is a Mexican-hat (difference of Gaussians) kernel:

$$W(x) = A_{exc} e^{-x^2/\sigma_{exc}^2} - A_{inh} e^{-x^2/\sigma_{inh}^2}$$

0.1

Write a Python function `mexican_hat_1d(x, A_exc, sigma_exc, A_inh, sigma_inh)` that computes the 1D interaction weight at distance x .

```
In [25]: def mexican_hat_1d(x, A_exc, sigma_exc, A_inh, sigma_inh):
    """
    Compute the 1D mexican-hat weight kernel.

    Parameters:
```

```

- x: array of positions
- A_exc: amplitude of the excitatory component
- sigma_exc: width of the excitatory component
- A_inh: amplitude of the inhibitory component
- sigma_inh: width of the inhibitory component
Returns:
- W: array of weights corresponding to positions in x
'''

exc = A_exc * np.exp(-x**2 / sigma_exc**2)
inh = A_inh * np.exp(-x**2 / sigma_inh**2)
W = exc - inh
return W

```

0.2

Plot the weight kernel for an array of $M = 100$ neurons centered around 0 (i.e., $x \in [-\frac{M}{2}, \frac{M}{2}]$, equally spaced), using parameters:

- $A_{exc} = 0.3$
- $\sigma_{exc} = 5.0$
- $A_{inh} = 0.2$
- $\sigma_{inh} = 10.0$

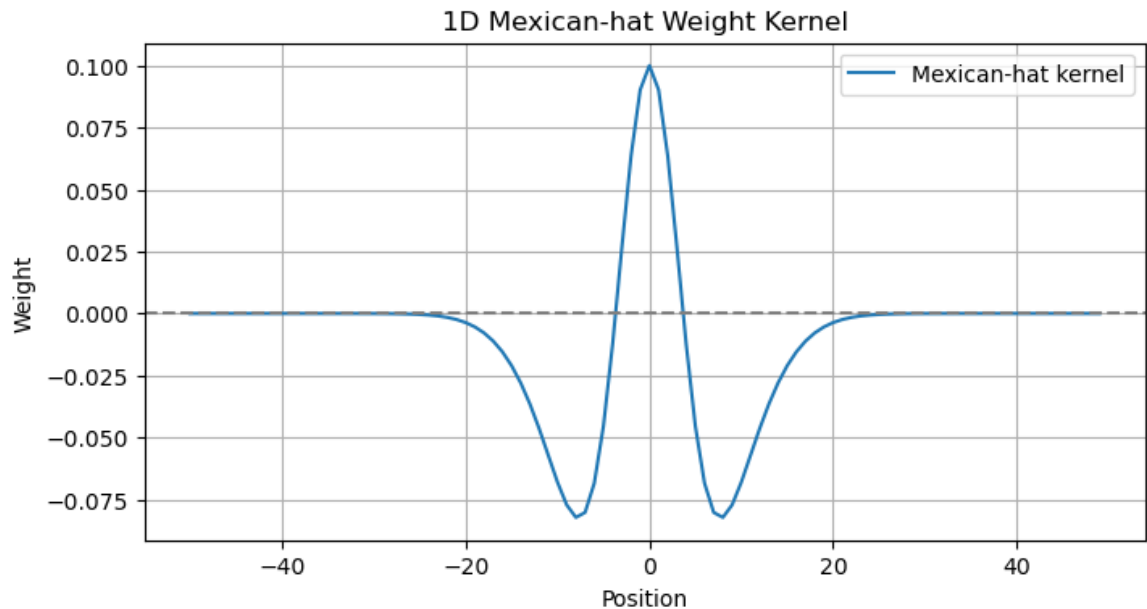
```

In [26]: M = 100
x = np.linspace(-M/2, M/2, M, endpoint=False) # M/2 not included
A_exc = 0.3
sigma_exc = 5.0
A_inh = 0.2
sigma_inh = 10.0

W = mexican_hat_1d(x, A_exc, sigma_exc, A_inh, sigma_inh) # Compute

# plot
plt.figure(figsize=(8, 4))
plt.plot(x, W, label='Mexican-hat kernel')
plt.axhline(0, color='gray', linestyle='--')
plt.title('1D Mexican-hat Weight Kernel')
plt.xlabel('Position')
plt.ylabel('Weight')
plt.legend()
plt.grid()
plt.show()

```



0.3

Explain which biological feature of cortical connectivity this kernel represents, in terms of excitation and inhibition.

Answer:

The mexican hat represents local excitation and lateral inhibition.

- Local excitation: Neurons that are close to each other excite one another, as we can see with the narrow Gaussian around $x = 0$.
- Lateral inhibition: Neurons that are not close enough to each other inhibit one another, as we can see with the negative Gaussian which is broader than the positive one (excitatory).

There is this difference because of the different sigmas (smaller for σ_{exc} than σ_{inh}). Biologically, this is due to excitatory pyramidal neurons make short-range synaptic connections with their neighbors and inhibitory interneurons inhibit neurons which are farther.

If we were to simply apply this connectivity scheme to M neurons, it is not clear how the neurons at the boundary should be connected to other neurons. Here, we assume the network relies on periodic boundary conditions. Mathematically, this means that the connectivity kernel is periodic, i.e.:

$$W(x + M) = W(x)$$

for every x . Equivalently, we could imagine that the M neurons lie on a ring.

To simulate the network, we will use the Forward Euler method with a time-step of $\Delta t = 1 \text{ ms}$:

$$s(x, t_{k+1}) = s(x, t_k) + \Delta t \cdot \frac{ds(x, t_k)}{dt}$$

0.4

Use the forward Euler method with timestep $\Delta t = 1 \text{ ms}$ to integrate the dynamics of this 1D network over 500 ms. Plot the evolution of the firing rate $r(x, t)$ of all neurons. (Suggestions: you could use a heat map to visualize this).

Parameters:

- $\tau = 10 \text{ ms}$
- $B(x, t) = 1.0$

Randomly initialize the potentials $s(x, 0)$ uniformly in $[0, 0.1]$.

Hint: To easily implement the convolution

$$\int W(x - x')r(x')dx'$$

discretely, you could first precompute the connectivity kernel as in Ex. 0.2 and then use `scipy.ndimage.convolve1d`, with proper mode to reflect the periodic boundary.

```
In [27]: # Euler
def forward_Euler(M, W, B, dt, tau, n_steps, seed=42):
    rng = np.random.default_rng(seed=seed)

    s = rng.uniform(0, 0.1, size=M) # potentials randomly initialize
    r_history = np.zeros((n_steps, M)) # firing rates over time

    for t in range(n_steps):
        r = np.maximum(0, s) # firing rates
        Irec = convolve1d(r, W, mode='wrap') # recurrent input
        s = s + dt/tau * (-s + Irec + B) # Euler
        r_history[t] = r

    return r_history

# heatmap
def plot_firing_rates(r_history, x, n_steps, seed=42):
    plt.figure(figsize=(8, 6))
    plt.imshow(r_history.T, aspect='auto', origin='lower', extent=[
    plt.colorbar(label='Firing Rate')
    plt.title(f'Firing Rates of Neurons Over Time (seed={seed})')
    plt.xlabel('Time (ms)')
    plt.ylabel('Position')
    plt.show()

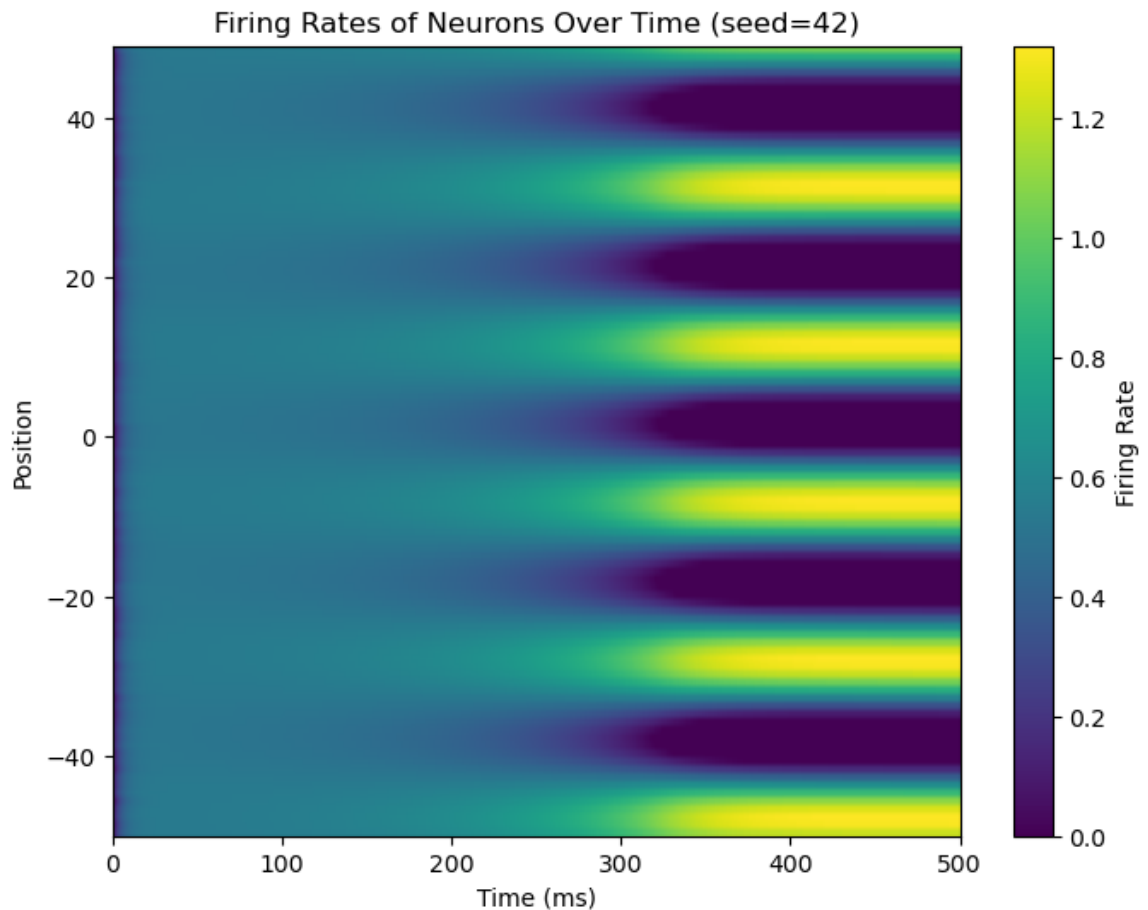
# Parameters
```

```

n_steps = 500
M = 100
tau = 10 # ms
dt = 1 # ms
B = 1.0
W = mexican_hat_1d(x, A_exc, sigma_exc, A_inh, sigma_inh) # Compute

# Simulate
r_history = forward_Euler(M=M, W=W, B=B, dt=dt, tau=tau, n_steps=n_steps)
plot_firing_rates(r_history, x, n_steps)

```



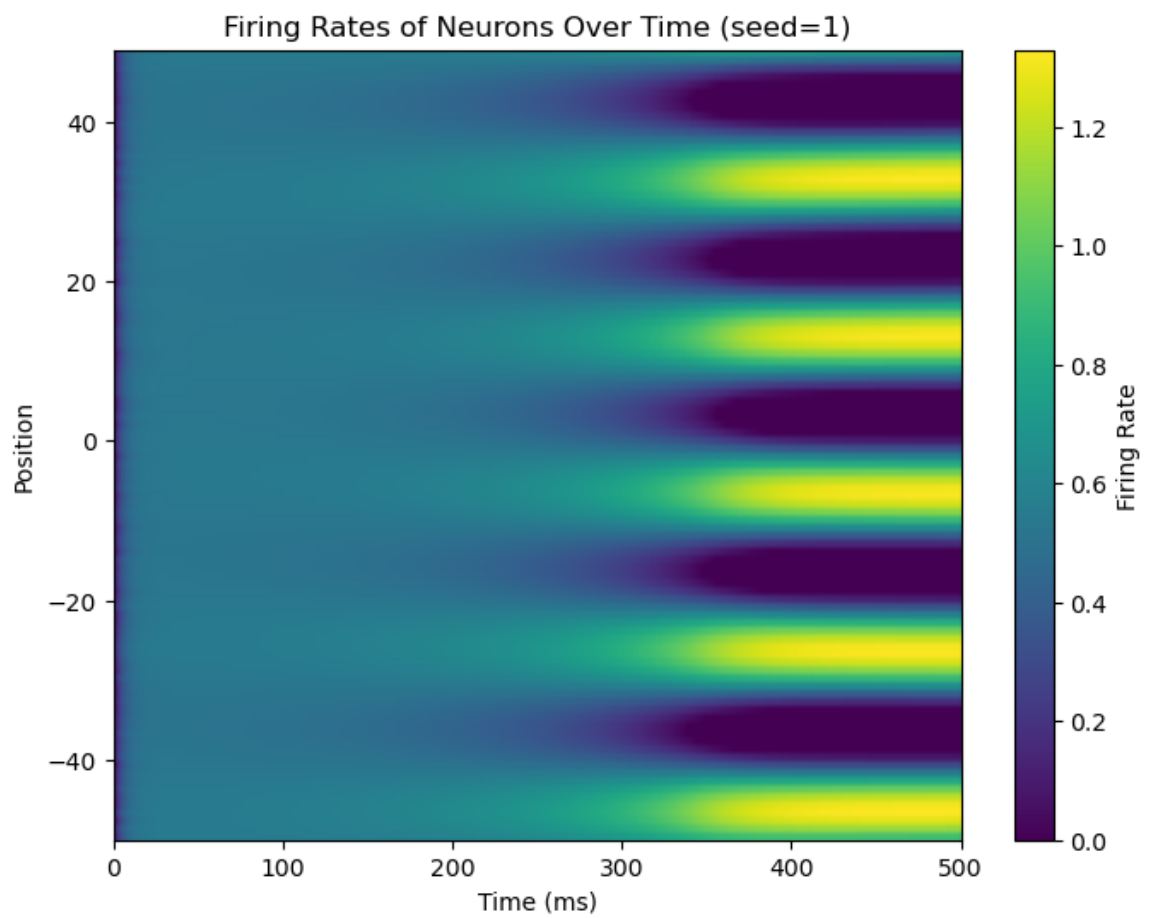
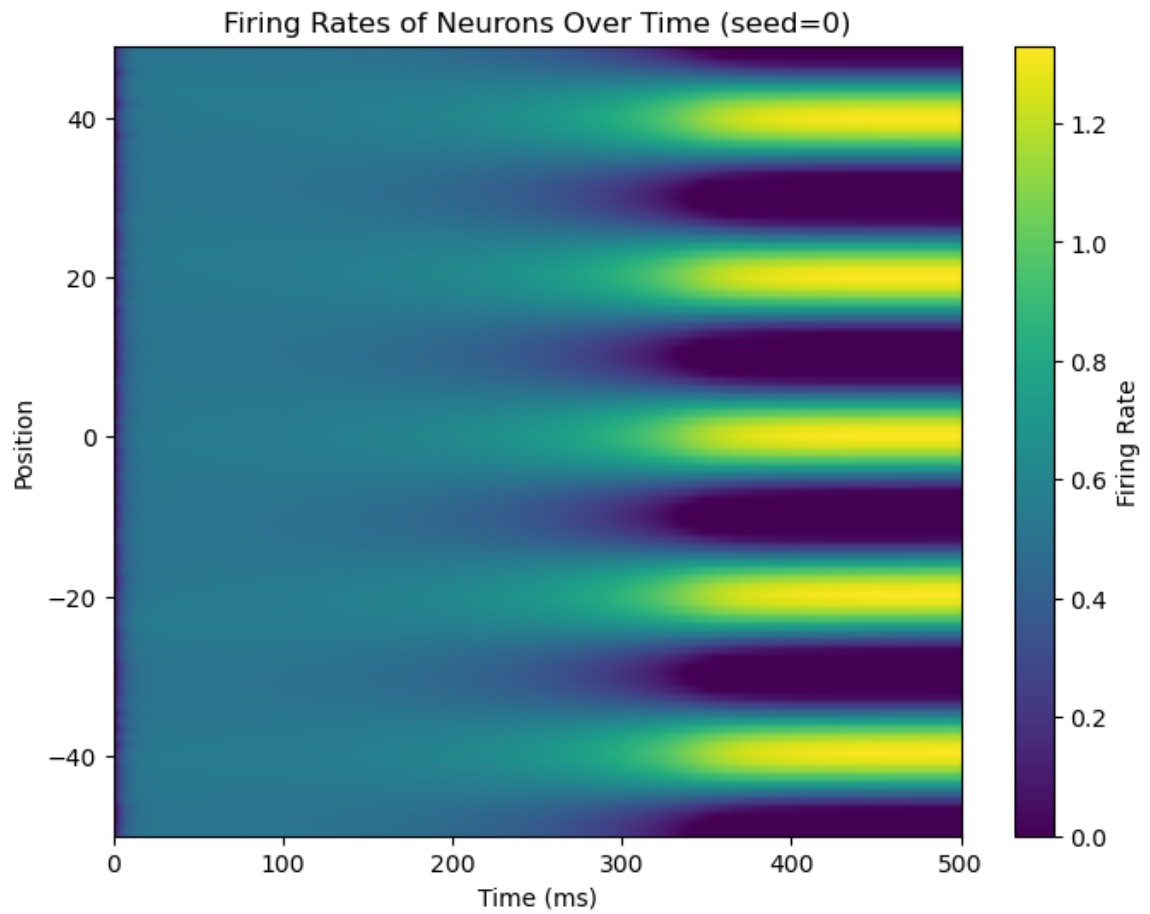
0.5

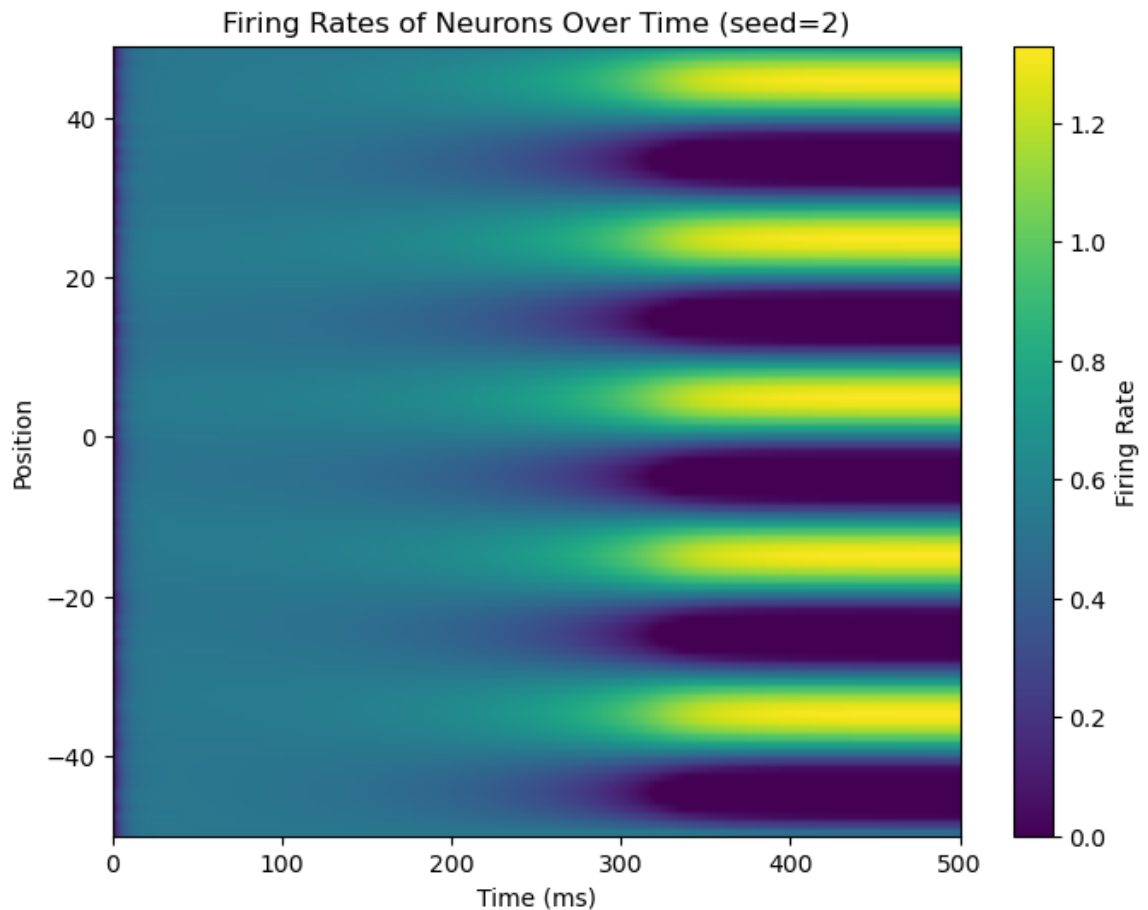
Describe the behavior of the network. Use 3 different random seeds to initialize the network. Does the behavior depend on the initialization of $s(x, 0)$?

```

In [28]: for seed in [0, 1, 2]:
          r_history = forward_Euler(M=M, W=W, B=B, dt=dt, tau=tau, n_steps=n_steps)
          plot_firing_rates(r_history, x, n_steps, seed=seed)

```





Answer:

The network self organizes from a random initial state to a stable pattern of periodic activity bumps. These bumps arrive around 200ms and stay constant after that. That reflects the attractor dynamic of the network. A bump corresponds to a group of neurons firing together via local excitation and neurons between the bumps are inhibited via lateral inhibition.

When we plot for 3 different seeds, we can see:

- Number of bumps and their spacing are identical. These parameters are determined by the sigmas.
- Amplitude of bumps is the same. Determined by A_{exc} and A_{inh} .
- Position of the bumps differs. So the position depends on the initialisation of $s(x, 0)$. The position is not fixed by the connectivity but by the history of the network.

Exercise 1 — 1D Velocity Integration

To track velocity, we include two sub-populations in the network: a "Right-shifting" population (R) and a "Left-shifting" population (L), each with $M = 100$ neurons. There are two neurons at each location x , one neuron for each population. The recurrent connectivity and external input are defined as

follows:

- The connection weights coming from the R population are shifted slightly to the right by a weight shift l , meaning a neuron at x' in R projects to a neuron at x with weight:

$$W_R(x - x') = W(x - x' - l)$$

- The connection weights coming from the L population are shifted to the left by l :

$$W_L(x - x') = W(x - x' + l)$$

- Each population receives its own external input, B_R and B_L , which is modulated asymmetrically by an external velocity signal $v(t)$:

$$B_R(x, t) = B_0(1 + \alpha v(t))$$

$$B_L(x, t) = B_0(1 - \alpha v(t))$$

Both populations project to each other as well as to themselves. The total recurrent input received by any neuron (whether it belongs to the R or L population) is the sum of inputs from both populations:

$$I_{rec}(x, t) = \int W_R(x - x') r_R(x', t) dx' + \int W_L(x - x') r_L(x', t) dx'$$

1.1

Write a simulation class or function for this 2-population integrator. Use the previous parameters, but set:

- $B_0 = 1.5$
- $\alpha = 0.5$
- $l = 1.0$

Run the simulation for 1000 ms with a timestep of 1 ms.

For the first 300 ms, set $v = 0$ to let the activity pattern stabilize, and you should observe stable bumps. From $t = 300$ ms to 700 ms, apply a constant velocity $v = 1.0$. For $t > 700$ ms, set $v = 0$.

Plot the sum of the firing rates:

$$r_{total}(x, t) = r_R(x, t) + r_L(x, t)$$

```
In [29]: B0 = 1.5
         alpha = 0.5
         l = 1.0
```

```

dt = 1.0 # ms
tau = 10.0 # ms
n_steps = 1000
x = np.linspace(-M/2, M/2, M, endpoint=False)

rng = np.random.default_rng(seed=42)
s_R = rng.uniform(0, 0.1, size=M) # potentials for R
s_L = rng.uniform(0, 0.1, size=M) # potentials for L

W_R = mexican_hat_1d(x - l, A_exc, sigma_exc, A_inh, sigma_inh)
W_L = mexican_hat_1d(x + l, A_exc, sigma_exc, A_inh, sigma_inh)

r_history_R = np.zeros((n_steps, M)) # to store firing rates of R
r_history_L = np.zeros((n_steps, M)) # to store firing rates of L

for t in range(n_steps):

    if t < 300:
        v = 0.0
    elif t <= 700:
        v = 1.0
    else:
        v = 0.0

    r_R = np.maximum(0, s_R) # firing rates for R
    r_L = np.maximum(0, s_L) # firing rates for L

    Irec = convolve1d(r_R, W_R, mode='wrap') + convolve1d(r_L, W_L,

    B_R = B0 * (1 + alpha * v) # external input for R
    B_L = B0 * (1 - alpha * v) # external input for L

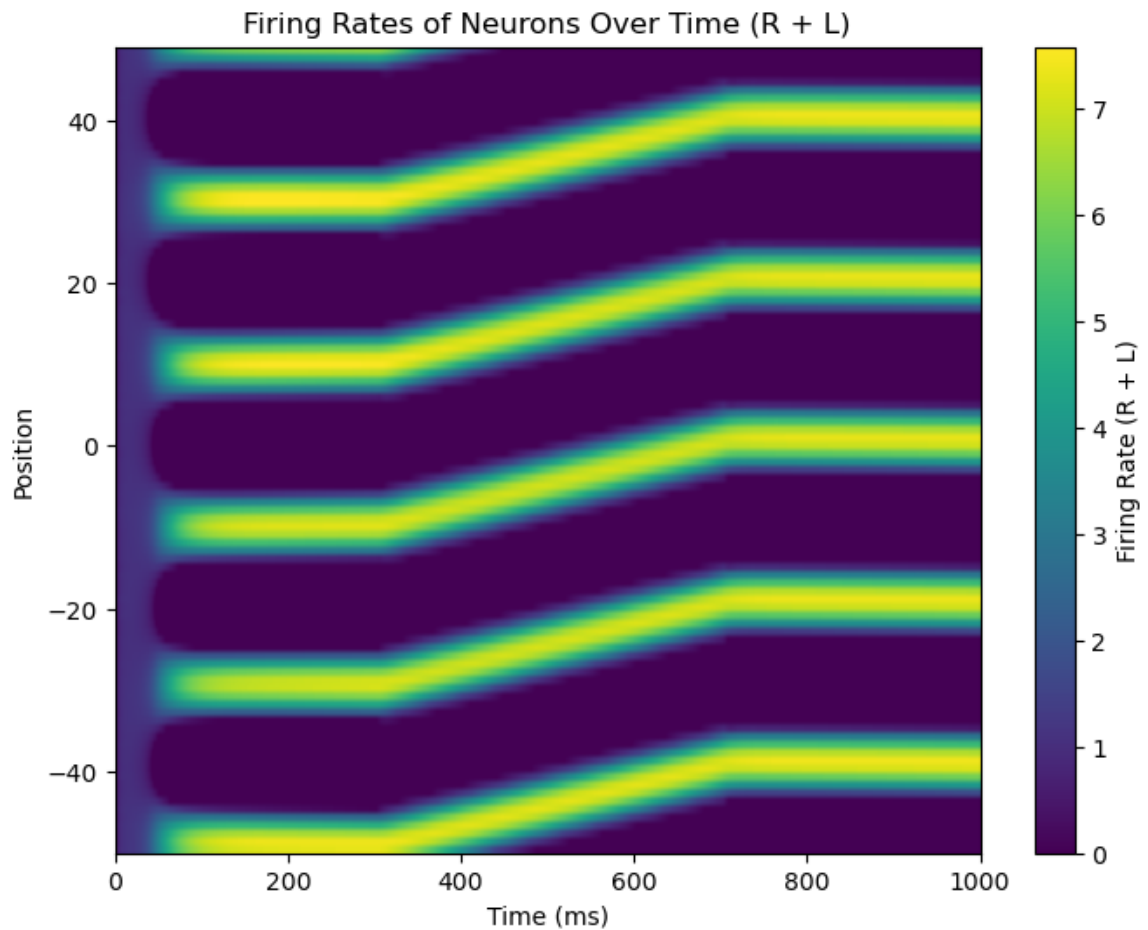
    s_R = s_R + dt/tau * (-s_R + Irec + B_R) # Euler for R
    s_L = s_L + dt/tau * (-s_L + Irec + B_L) # Euler for L

    r_history_R[t] = r_R # store firing rates for R
    r_history_L[t] = r_L # store firing rates for L

r_history_tot = r_history_R + r_history_L

# plot sum of firing rates
plt.figure(figsize=(8, 6))
plt.imshow(r_history_tot.T, aspect='auto', origin='lower', extent=[
plt.colorbar(label='Firing Rate (R + L)')
plt.title('Firing Rates of Neurons Over Time (R + L)')
plt.xlabel('Time (ms)')
plt.ylabel('Position')
plt.show()

```



1.2

Explain how the asymmetric scaling of the external inputs leads to shifting bumps in response to a velocity signal, based on the interactions between the two populations.

Answer:

With a positive velocity v , the external inputs are asymmetric:

$B_R = B_0 * (1 + \alpha * v) > B_L = B_0 * (1 - \alpha * v)$. Since B influences the activity of a population, the R population will be more active than the L.

Since the R's connectivity kernel W_R is shifted to the right by l , a more active R population sends stronger recurrent excitation to the right because its kernel is centered to the right of each neuron. L is less active and then sends weaker excitation to the left.

Because of this asymmetry, the activity bumps are drifted to the right (up in the graph). The drift speed is proportional to v : larger the v is, larger the asymmetry is and then faster the bumps move. With $v = 0$, the populations are equally active and the bump are constant in position across time.

1.3

To show that the bump acts as a velocity integrator, we apply a velocity signal v of varying magnitudes over time.

Sample 10 velocities in the interval:

$$[-2, -0.5] \cup [0.5, 2]$$

The velocity changes every 300 timesteps.

Plot the evolution of the bump and the velocity signal side by side. Describe your observations.

```
In [30]: rng_v = np.random.default_rng(seed=42)
magnitudes = rng_v.uniform(0.5, 2, size=10) # between 0.5 and 2
signs = rng_v.choice([-1, 1], size=10) # randomly choose signs
velocities = magnitudes * signs # final velocities
v_signal = np.repeat(velocities, 300)
n_steps = 3000 # 10 velocities * 300 steps each

s_R_v = rng_v.uniform(0, 0.1, size=M) # potentials for R
s_L_v = rng_v.uniform(0, 0.1, size=M) # potentials for L
r_history_R_v = np.zeros((n_steps, M)) # to store firing rates of R
r_history_L_v = np.zeros((n_steps, M)) # to store firing rates of
r_history_tot_v = np.zeros((n_steps, M)) # to store firing rates fo

for t in range(n_steps):

    v = v_signal[t] # get the current velocity

    r_R = np.maximum(0, s_R_v) # firing rates for R
    r_L = np.maximum(0, s_L_v) # firing rates for L

    Irec = convolve1d(r_R, W_R, mode='wrap') + convolve1d(r_L, W_L,

    B_R = B0 * (1 + alpha * v) # external input for R
    B_L = B0 * (1 - alpha * v) # external input for L

    s_R_v = s_R_v + dt/tau * (-s_R_v + Irec + B_R) # Euler for R
    s_L_v = s_L_v + dt/tau * (-s_L_v + Irec + B_L) # Euler for L

    r_history_R_v[t] = r_R # store firing rates for R
    r_history_L_v[t] = r_L # store firing rates for L

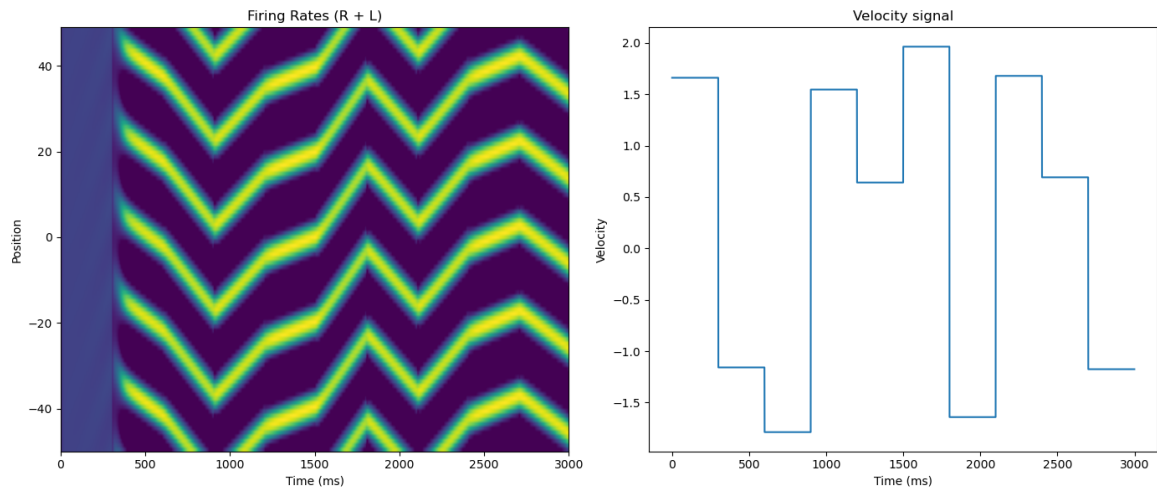
r_history_tot_v = r_history_R_v + r_history_L_v # store total firing

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# heatmap
axes[0].imshow(r_history_tot_v.T, aspect='auto', origin='lower',
               extent=[0, n_steps, x[0], x[-1]])
axes[0].set_xlabel('Time (ms)')
axes[0].set_ylabel('Position')
axes[0].set_title('Firing Rates (R + L)')
```

```
# velocity signal
axes[1].plot(v_signal)
axes[1].set_xlabel('Time (ms)')
axes[1].set_ylabel('Velocity')
axes[1].set_title('Velocity signal')

plt.tight_layout()
plt.show()
```



Answer:

We can see that before $t = 300$ ms, the system is not yet stabilized. The bumps emerge progressively from the random initial state through recurrent interactions. And then when

- $v > 0$: bumps drift to the right (up in the graph)
- $v < 0$: bumps drift to the left (down in the graph)

And larger the speed is (absolutely: $|v|$), faster the drift is (steeper the slope is). The network is therefore a velocity integrator: the bump position encodes the integral of the velocity signal over time, in other words the displacement.

1.4

Devise a method to compute the drifting speed of the 1D bump pattern. Use this method to extract the bump drifting speed at different input velocities v sampled in 1.3.

Plot the drifting speed versus the actual input velocity.

Can you see a linear relationship between the two?

```
In [31]: # extract bump's position
bump_positions = np.zeros(n_steps)
for t in range(n_steps):
    bump_positions[t] = x[int(np.argmax(r_history_tot_v[t]))]
```

```

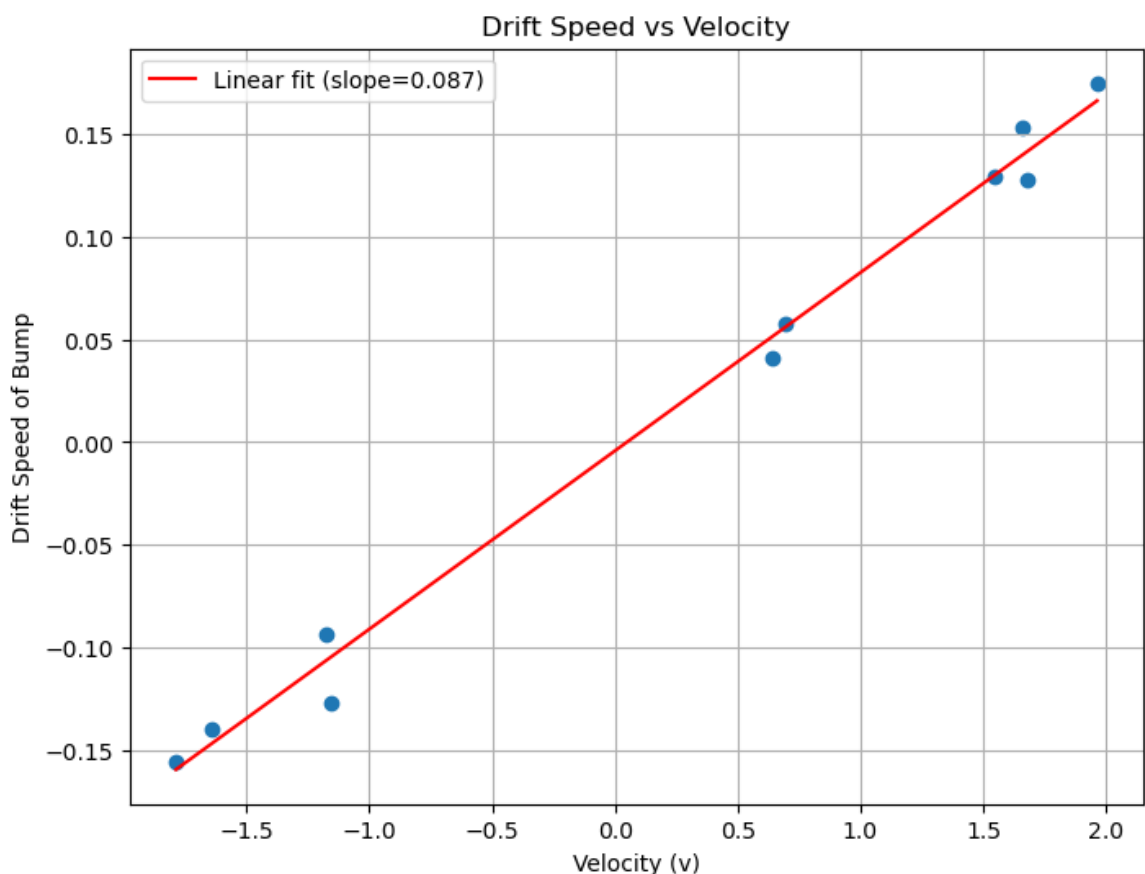
# line plot of bump position
drift_speeds = np.zeros(10)
for i in range(10):
    segment = bump_positions[i*300:(i+1)*300]
    segment = np.unwrap(segment, period=M) # unwrap to handle circu
    t_seg = np.arange(300)
    slope, _ = np.polyfit(t_seg, segment, 1) # linear fit
    drift_speeds[i] = slope

# plot drift_speeds vs velocities
plt.figure(figsize=(8, 6))
plt.scatter(velocities, drift_speeds)

# plot linear fit
coeffs = np.polyfit(velocities, drift_speeds, 1)
fit_line = np.polyval(coeffs, velocities)
plt.plot(np.sort(velocities), np.polyval(coeffs, np.sort(velocities)
plt.legend()

plt.xlabel('Velocity (v)')
plt.ylabel('Drift Speed of Bump')
plt.title('Drift Speed vs Velocity')
plt.grid()
plt.show()

```



Answer:

Yes we can see a clear linear relationship between the drift speed of the bump and the input velocity v . We plotted the linear fit and we found a slope of

0.087 which means for each unit of vitess change, the drift speed change of 0.087. And we can also see that the intercept is around $(0, 0)$ which confirm, together with the positive slope, that a negative input velocity induces a negative drift speed (to the left) and a positive input velocity induces a positive drift speed (to the right). And also that a without input velocity, the bumps do not drift.

Exercise 2 — Extending to 2D Grid Cells

We now extend to a 2D sheet of neurons. Each neuron has a 2D position $\vec{x} = (x, y)$. Then equations (1) and (2) can be directly extended for the 2D case by using $\vec{x}$ and $\vec{x}'$ to replace x and x' .

The Mexican-hat connectivity in 2D is defined using the Euclidean norm of the vector:

$$||\vec{x}|| = \sqrt{x^2 + y^2}$$

$$W(\vec{x}) = A_{exc}e^{-||\vec{x}||^2/\sigma_{exc}^2} - A_{inh}e^{-||\vec{x}||^2/\sigma_{inh}^2}$$

2.1

Consider an $M \times M$ array of neurons ($M = 40$, so 1600 neurons), plot the mexican-hat weight kernel $W(\vec{x})$ centered around 0 in a 2D heatmap (i.e., $x, y \in [-\frac{M}{2}, \frac{M}{2})$, equally spaced). Use parameters:

- $A_{exc} = 1.0$
- $A_{inh} = 1.0$
- $\sigma_{exc} = 4.8$
- $\sigma_{inh} = 5.0$

```
In [32]: A_exc = 1.0
A_inh = 1.0
sigma_exc = 4.8
sigma_inh = 5.0

M = 40

x = np.linspace(-M/2, M/2, M, endpoint=False)
y = np.linspace(-M/2, M/2, M, endpoint=False)
X, Y = np.meshgrid(x, y)

def mexican_hat_2d(X, Y, A_exc, sigma_exc, A_inh, sigma_inh):
    """
    Compute the 2D mexican-hat weight kernel.

    Parameters:
```

```

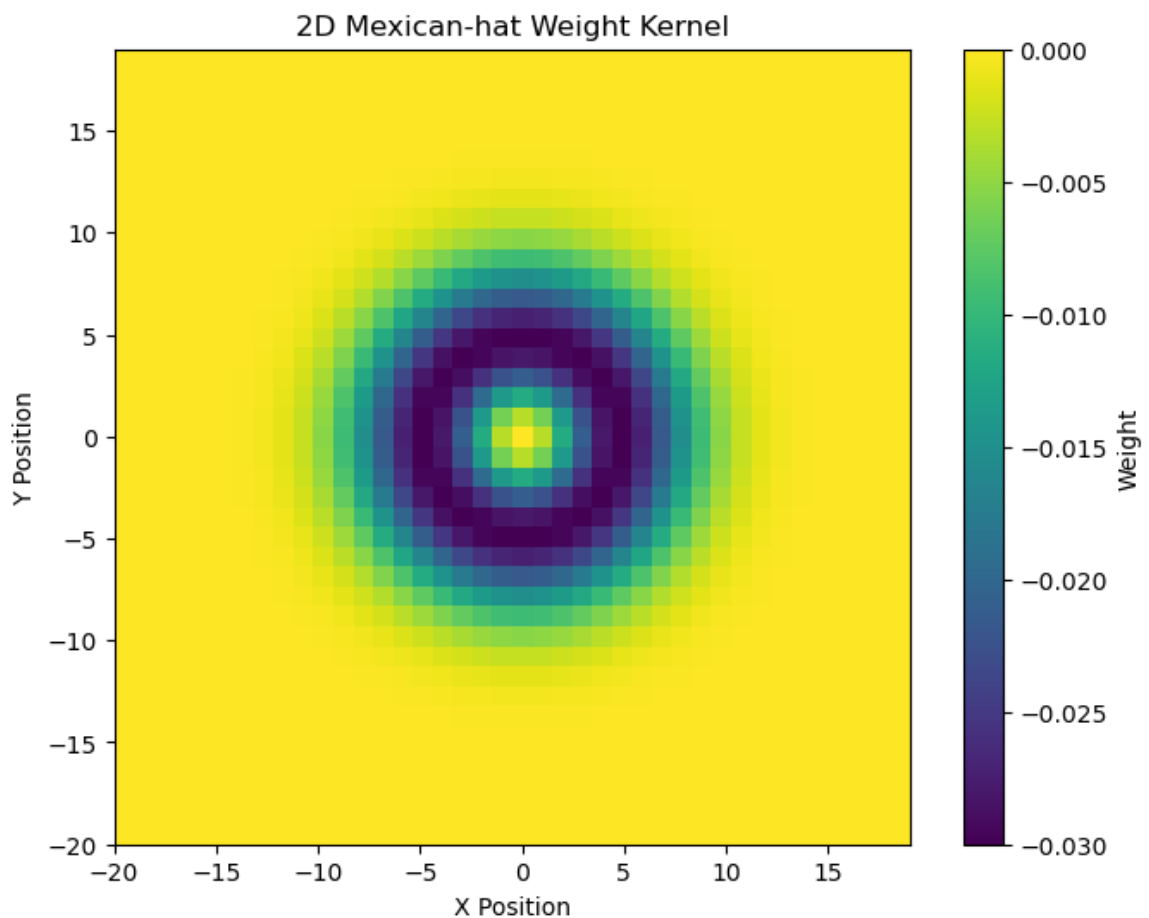
- X, Y: 2D arrays of positions
- A_exc: amplitude of the excitatory component
- sigma_exc: width of the excitatory component
- A_inh: amplitude of the inhibitory component
- sigma_inh: width of the inhibitory component
Returns:
- W: 2D array of weights corresponding to positions in X, Y
...

R = np.sqrt(X**2 + Y**2)
exc = A_exc * np.exp(-R**2 / sigma_exc**2)
inh = A_inh * np.exp(-R**2 / sigma_inh**2)
W = exc - inh

return W

# plot
W_2d = mexican_hat_2d(X, Y, A_exc, sigma_exc, A_inh, sigma_inh)
plt.figure(figsize=(8, 6))
plt.imshow(W_2d, extent=[x[0], x[-1], y[0], y[-1]], origin='lower')
plt.colorbar(label='Weight')
plt.title('2D Mexican-hat Weight Kernel')
plt.xlabel('X Position')
plt.ylabel('Y Position')
plt.show()

```



To model movement in the 2D space, we use 4 distinct populations: North (N), South (S), East (E), and West (W), each with 2D Mexican-hat kernels shifted by a weight shift l along their respective preferred axes:

$$W_N(\vec{x}) = W(\vec{x} - (0, l)) \quad W_S(\vec{x}) = W(\vec{x} + (0, l))$$

$$W_E(\vec{x}) = W(\vec{x} - (l, 0)) \quad W_W(\vec{x}) = W(\vec{x} + (l, 0))$$

The feedforward inputs are modulated by a 2D velocity vector

$$\vec{v}(t) = (v_x(t), v_y(t)):$$

$$B_N = B_0(1 + \alpha v_y(t)) \quad B_S = B_0(1 - \alpha v_y(t))$$

$$B_E = B_0(1 + \alpha v_x(t)) \quad B_W = B_0(1 - \alpha v_x(t))$$

2.2

Implement the 2D model. Create an $M \times M$ array of neurons ($M = 40$). Use the above mexican-hat kernel as 2D connectivity with periodic boundary (topologically, connecting the boundaries of a 2D sheet means that the neurons lie on a torus).

Simulate with $\vec{v} = (0, 0)$ until the pattern stabilizes (e.g., 400 ms). Plot a heatmap of the total 2D firing rate:

$$r_{total}(x, y)$$

(sum of all 4 populations) at the final timestep.

Do you see a grid pattern?

Use parameters:

- $\tau = 10 \text{ ms}$
- $\Delta t = 1 \text{ ms}$
- $l = 1.0$
- $\alpha = 0.1$
- $B_0 = 1.0$

```
In [33]: tau = 10.0 # ms
dt = 1.0 # ms
l = 1.0
alpha = 0.1
B0 = 1.0

n_steps = 400

# initial potentials
rng = np.random.default_rng(seed=42)
s_N = rng.uniform(0, 0.1, size=(M, M))
s_S = rng.uniform(0, 0.1, size=(M, M))
s_E = rng.uniform(0, 0.1, size=(M, M))
s_W = rng.uniform(0, 0.1, size=(M, M))

r_N_history = np.zeros((n_steps, M, M))
```

```

r_S_history = np.zeros((n_steps, M, M))
r_E_history = np.zeros((n_steps, M, M))
r_W_history = np.zeros((n_steps, M, M))
r_history_tot_4pop = np.zeros((n_steps, M, M))

W_N = mexican_hat_2d(X, Y-l, A_exc, sigma_exc, A_inh, sigma_inh)
W_S = mexican_hat_2d(X, Y+l, A_exc, sigma_exc, A_inh, sigma_inh)
W_E = mexican_hat_2d(X-l, Y, A_exc, sigma_exc, A_inh, sigma_inh)
W_W = mexican_hat_2d(X+l, Y, A_exc, sigma_exc, A_inh, sigma_inh)

v_x, v_y = 0.0, 0.0

for t in range(n_steps):

    r_N = np.maximum(0, s_N)
    r_S = np.maximum(0, s_S)
    r_E = np.maximum(0, s_E)
    r_W = np.maximum(0, s_W)

    Irec = convolve(r_N, W_N, mode='wrap') + convolve(r_S, W_S, mode='wrap') + convolve(r_E, W_E, mode='wrap') + convolve(r_W, W_W, mode='wrap')

    B_N = B0 * (1 + alpha * v_y)
    B_S = B0 * (1 - alpha * v_y)
    B_E = B0 * (1 + alpha * v_x)
    B_W = B0 * (1 - alpha * v_x)

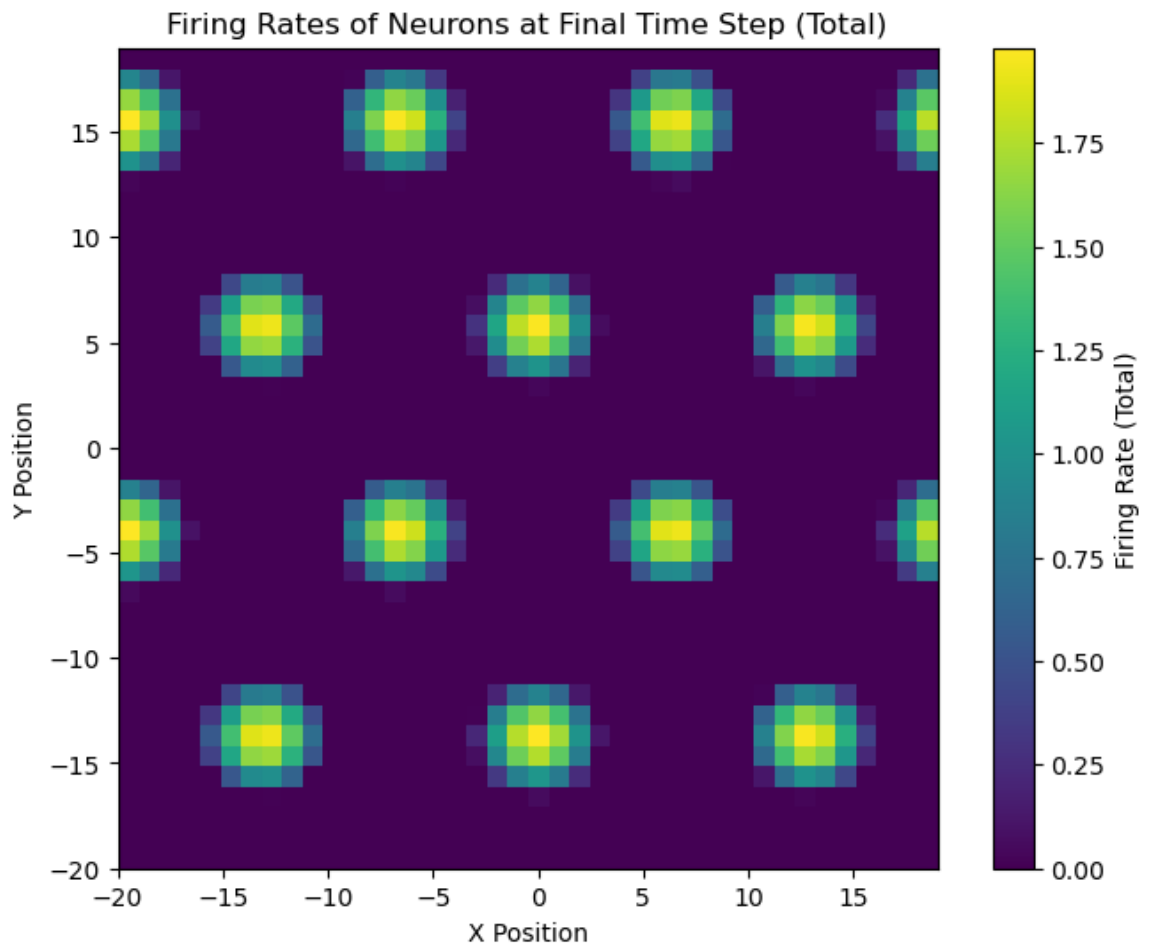
    s_N += dt/tau * (-s_N + Irec + B_N)
    s_S += dt/tau * (-s_S + Irec + B_S)
    s_E += dt/tau * (-s_E + Irec + B_E)
    s_W += dt/tau * (-s_W + Irec + B_W)

    r_N_history[t] = r_N
    r_S_history[t] = r_S
    r_E_history[t] = r_E
    r_W_history[t] = r_W

r_history_tot_4pop = r_N_history + r_S_history + r_E_history + r_W_history

# plot 2d heatmap of total firing rates
plt.figure(figsize=(8, 6))
plt.imshow(r_history_tot_4pop[-1], extent=[x[0], x[-1], y[0], y[-1]])
plt.colorbar(label='Firing Rate (Total)')
plt.title('Firing Rates of Neurons at Final Time Step (Total)')
plt.xlabel('X Position')
plt.ylabel('Y Position')
plt.show()

```



Answer:

We can clearly see a grid pattern, which is a stable periodic pattern of activity across the neural sheet. The bumps self-organize from the random initial state into a regular 2D lattice and this lattice is separated by silent regions. This is due to the recurrent connectivity. Indeed, local excitation groups nearby neurons into bumps and lateral inhibition inhibit surrounding activity.

2.3

To test that your grid maps act as a continuous 2D integrator, apply constant velocity vectors in at least 3 different directions (e.g., purely horizontal, purely vertical, and diagonal).

For each velocity direction:

- run your network without any velocity for 300 ms
- then apply input velocity for 300 ms

Visualize the total 2D firing rate $r_{total}(x, y)$ side-by-side at 3 distinct time points (e.g., $t = 300$, $t = 450$, $t = 600$ ms).

Describe the integration motion resulting from your velocity input vectors.

Hint: Convolution over the 2D space can be easily implemented using `scipy.ndimage.convolve2d` and setting the boundary conditions accordingly. Personal warning: `scipy.ndimage.convolve2d` doesn't exist, we will use `scipy.ndimage.convolve`.

```
In [34]: directions = ['Horizontal', 'Vertical', 'Diagonal']

# compute weight kernels for each population
W_N = mexican_hat_2d(X, Y-l, A_exc, sigma_exc, A_inh, sigma_inh)
W_S = mexican_hat_2d(X, Y+l, A_exc, sigma_exc, A_inh, sigma_inh)
W_E = mexican_hat_2d(X-l, Y, A_exc, sigma_exc, A_inh, sigma_inh)
W_W = mexican_hat_2d(X+l, Y, A_exc, sigma_exc, A_inh, sigma_inh)

for direction in directions:
    # re-initialize potentials and histories for each direction
    r_N_history = np.zeros((600, M, M))
    r_S_history = np.zeros((600, M, M))
    r_E_history = np.zeros((600, M, M))
    r_W_history = np.zeros((600, M, M))
    r_history_tot_4pop = np.zeros((600, M, M))

    s_N = rng.uniform(0, 0.1, size=(M, M))
    s_S = rng.uniform(0, 0.1, size=(M, M))
    s_E = rng.uniform(0, 0.1, size=(M, M))
    s_W = rng.uniform(0, 0.1, size=(M, M))

    for t in range(0, 600):
        if t < 300:
            v_x, v_y = 0.0, 0.0
        else:
            if direction == 'Horizontal':
                v_x, v_y = 1.0, 0.0
            elif direction == 'Vertical':
                v_x, v_y = 0.0, 1.0
            elif direction == 'Diagonal':
                v_x, v_y = 1.0, 1.0

        r_N = np.maximum(0, s_N)
        r_S = np.maximum(0, s_S)
        r_E = np.maximum(0, s_E)
        r_W = np.maximum(0, s_W)

        Irec = convolve(r_N, W_N, mode='wrap') + convolve(r_S, W_S,

B_N = B0 * (1 + alpha * v_y)
B_S = B0 * (1 - alpha * v_y)
B_E = B0 * (1 + alpha * v_x)
B_W = B0 * (1 - alpha * v_x)

s_N += dt/tau * (-s_N + Irec + B_N)
s_S += dt/tau * (-s_S + Irec + B_S)
s_E += dt/tau * (-s_E + Irec + B_E)
s_W += dt/tau * (-s_W + Irec + B_W)

r_N_history[t] = r_N
```

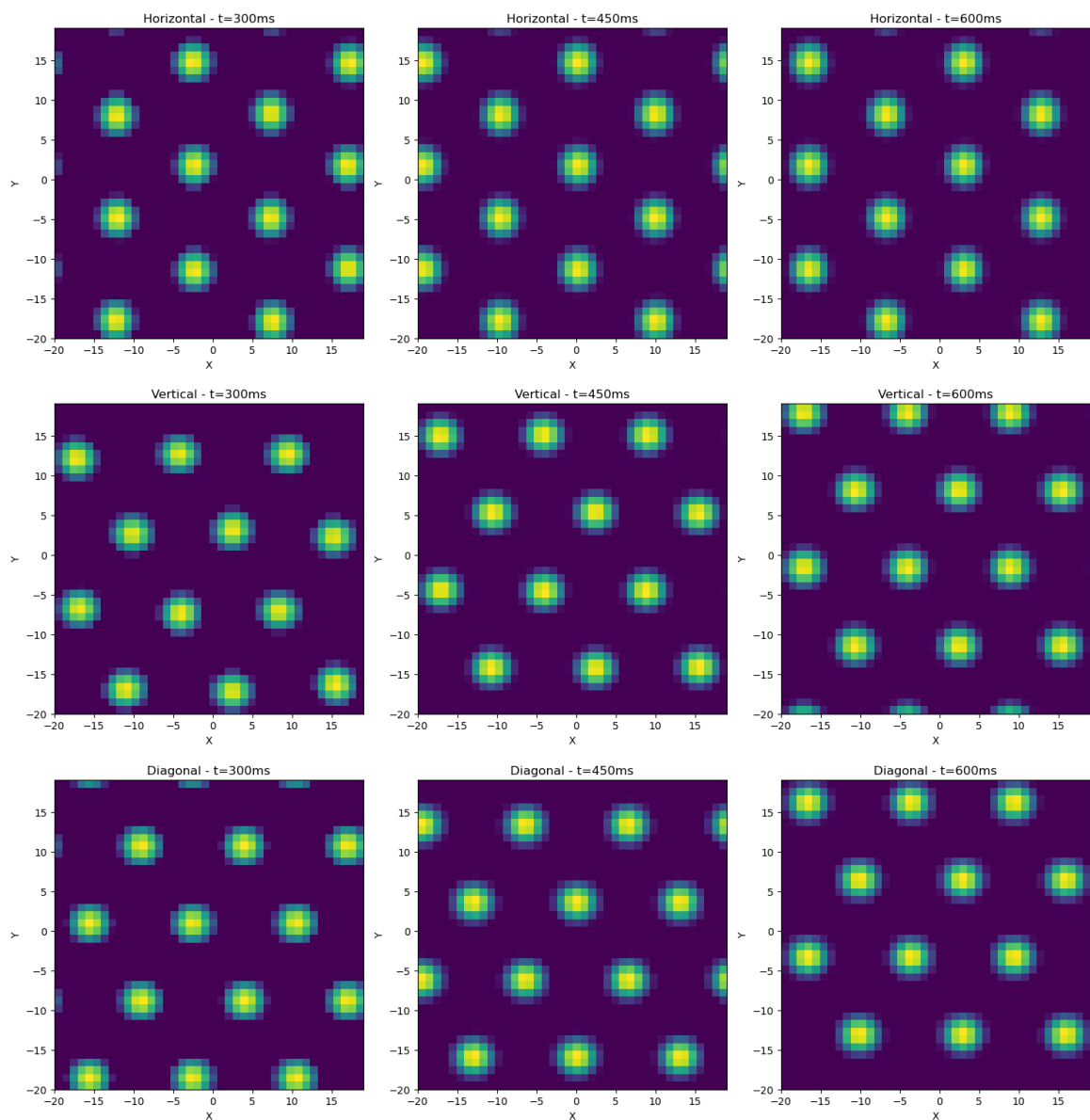
```

r_S_history[t] = r_S
r_E_history[t] = r_E
r_W_history[t] = r_W

r_history_tot_4pop = r_N_history + r_S_history + r_E_history +

# 3 snapshots side by side at t=300ms, t=450ms, t=600ms
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for ax, t_snap, label in zip(axes, [299, 449, 599], ['t=300ms',
ax.imshow(r_history_tot_4pop[t_snap], extent=[x[0], x[-1],
ax.set_title(f'{direction} - {label}'))
ax.set_xlabel('X')
ax.set_ylabel('Y')
plt.tight_layout()
plt.show()

```



Answer:

We can see that the direction of the velocity induces the direction of the drift.

Indeed,

- Horizontal velocity (1.0, 0): The bumps drift to the right along the x-axis
- Vertical velocity (0, 1.0): The bumps drift upward along the y-axis
- Diagonal velocity (1.0, 1.0): The bumps drift upward and to the right

Since the grid patterns translates as a whole and without deforming, we can say that the 2D network acts as a continuous integrator of the 2D velocity vector.

2.4 (Bonus)

Generate an input pattern with changing velocity.

Start with a velocity pointing in a random direction, with a speed sampled uniformly in $[1, 2]$. Sample a new velocity every 100 ms, and you could smooth the velocity when there is a change.

Estimate the input velocity $\vec{v}$ from the drifting of the grid patterns.

Does it track well the input velocity?

```
In [35]: M = 40
# compute weight kernels for each population
W_N = mexican_hat_2d(X, Y-l, A_exc, sigma_exc, A_inh, sigma_inh)
W_S = mexican_hat_2d(X, Y+l, A_exc, sigma_exc, A_inh, sigma_inh)
W_E = mexican_hat_2d(X-l, Y, A_exc, sigma_exc, A_inh, sigma_inh)
W_W = mexican_hat_2d(X+l, Y, A_exc, sigma_exc, A_inh, sigma_inh)

# initialize potentials and histories
r_N_history = np.zeros((600, M, M))
r_S_history = np.zeros((600, M, M))
r_E_history = np.zeros((600, M, M))
r_W_history = np.zeros((600, M, M))
r_history_tot_4pop = np.zeros((600, M, M))

s_N = rng.uniform(0, 0.1, size=(M, M))
s_S = rng.uniform(0, 0.1, size=(M, M))
s_E = rng.uniform(0, 0.1, size=(M, M))
s_W = rng.uniform(0, 0.1, size=(M, M))

# smooth transition parameters
smooth_window = 10 # ms
transition = False
old_v_x, old_v_y = 0.0, 0.0
new_v_x, new_v_y = 0.0, 0.0
v_x_signal = []
v_y_signal = []

for t in range(0, 600):
    if t < 299:
        v_x, v_y = 0.0, 0.0
```

```

else:
    if (t+1) % 100 == 0:
        transition = True
        old_v_x, old_v_y = v_x, v_y
        base_velocity = rng.uniform(1.0, 2.0)
        angle = rng.uniform(0, 2*np.pi)
        new_v_x, new_v_y = base_velocity * np.cos(angle), base_
    if (t+1) % 100 < smooth_window and transition:
        v_x = old_v_x + (new_v_x - old_v_x) * ((t+1) % 100) / sm
        v_y = old_v_y + (new_v_y - old_v_y) * ((t+1) % 100) / sm
    else:
        v_x, v_y = new_v_x, new_v_y
        transition = False
    v_x_signal.append(v_x)
    v_y_signal.append(v_y)

r_N = np.maximum(0, s_N)
r_S = np.maximum(0, s_S)
r_E = np.maximum(0, s_E)
r_W = np.maximum(0, s_W)

Irec = convolve(r_N, W_N, mode='wrap') + convolve(r_S, W_S, mode=

B_N = B0 * (1 + alpha * v_y)
B_S = B0 * (1 - alpha * v_y)
B_E = B0 * (1 + alpha * v_x)
B_W = B0 * (1 - alpha * v_x)

s_N += dt/tau * (-s_N + Irec + B_N)
s_S += dt/tau * (-s_S + Irec + B_S)
s_E += dt/tau * (-s_E + Irec + B_E)
s_W += dt/tau * (-s_W + Irec + B_W)

r_N_history[t] = r_N
r_S_history[t] = r_S
r_E_history[t] = r_E
r_W_history[t] = r_W

r_history_tot_4pop = r_N_history + r_S_history + r_E_history + r_W_

search_r = 5
iy, ix = np.unravel_index(np.argmax(r_history_tot_4pop[300]), (M, M
bump_x = np.zeros(300)
bump_y = np.zeros(300)

for t in range(300):
    frame = r_history_tot_4pop[300 + t]
    iy_idx = np.arange(iy - search_r, iy + search_r + 1) % M
    ix_idx = np.arange(ix - search_r, ix + search_r + 1) % M
    local = frame[np.ix_(iy_idx, ix_idx)]

    s = local.sum()
    cy = np.sum(np.arange(2*search_r+1)[: , None] * local) / s
    cx = np.sum(np.arange(2*search_r+1)[None, :] * local) / s

    iy = iy_idx[int(round(cy))]

```

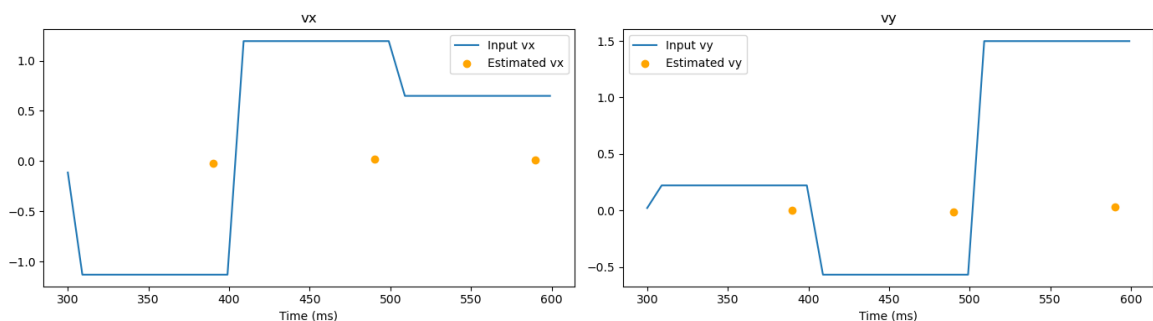
```

ix = ix_idx[int(round(cx))]
bump_y[t] = y[0] + ((iy_idx[0] + cy) % M) * (y[1]-y[0])
bump_x[t] = x[0] + ((ix_idx[0] + cx) % M) * (x[1]-x[0])

# Polyfit sur fenêtrées de 50ms (identique à 1.4)
est_times = [390, 490, 590]
est_vx, est_vy = [], []
win = 50
t_win = np.arange(win)
for t_est in est_times:
    i_end = t_est - 300
    i_start = i_end - win
    seg_x = np.unwrap(bump_x[i_start:i_end], period=M)
    seg_y = np.unwrap(bump_y[i_start:i_end], period=M)
    slope_x, _ = np.polyfit(t_win, seg_x, 1)
    slope_y, _ = np.polyfit(t_win, seg_y, 1)
    est_vx.append(slope_x)
    est_vy.append(slope_y)

t_axis = np.arange(300, 600)
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
axes[0].plot(t_axis, v_x_signal[1:], label='Input vx')
axes[0].scatter(est_times, est_vx, color='orange', zorder=5, label=
axes[0].legend(); axes[0].set_title('vx'); axes[0].set_xlabel('Time
axes[1].plot(t_axis, v_y_signal[1:], label='Input vy')
axes[1].scatter(est_times, est_vy, color='orange', zorder=5, label=
axes[1].legend(); axes[1].set_title('vy'); axes[1].set_xlabel('Time
plt.tight_layout()
plt.show()

```



Exercise 3 — Aperiodic Boundaries

Periodic boundaries (a torus connection topology) are biologically implausible as raw physical tissue does not wrap around itself. The edges of the neural tissue must be considered.

We now consider the aperiodic boundary: if the source neuron of a connection weight exceeds the boundary, we just omit the connection.

Mathematically, we define a strict boundary for integrals in the expression of $I_{rec}(\vec{x}, t)$:

$$I_{rec}(\vec{x}, t) = \int_0^M \int_0^M W(\vec{x} - \vec{x}') r(\vec{x}', t) dx' dy'$$

If you are using `scipy.ndimage.convolve2d`, this could be efficiently implemented by setting `mode='constant'`.

3.1

Repeat the simulation from Ex. 2.2 but with aperiodic boundaries. Describe what you see.

```
In [36]: A_exc = 1.0
A_inh = 1.0
sigma_exc = 4.8
sigma_inh = 5.0

M = 40

x = np.linspace(-M/2, M/2, M, endpoint=False)
y = np.linspace(-M/2, M/2, M, endpoint=False)
X, Y = np.meshgrid(x, y)

tau = 10.0 # ms
dt = 1.0 # ms
l = 1.0
alpha = 0.1
B0 = 1.0

n_steps = 400

# initial potentials
rng = np.random.default_rng(seed=42)
s_N = rng.uniform(0, 0.1, size=(M, M))
s_S = rng.uniform(0, 0.1, size=(M, M))
s_E = rng.uniform(0, 0.1, size=(M, M))
s_W = rng.uniform(0, 0.1, size=(M, M))

r_N_history = np.zeros((n_steps, M, M))
r_S_history = np.zeros((n_steps, M, M))
r_E_history = np.zeros((n_steps, M, M))
r_W_history = np.zeros((n_steps, M, M))
r_history_tot_4pop = np.zeros((n_steps, M, M))

W_N = mexican_hat_2d(X, Y-l, A_exc, sigma_exc, A_inh, sigma_inh)
W_S = mexican_hat_2d(X, Y+l, A_exc, sigma_exc, A_inh, sigma_inh)
W_E = mexican_hat_2d(X-l, Y, A_exc, sigma_exc, A_inh, sigma_inh)
W_W = mexican_hat_2d(X+l, Y, A_exc, sigma_exc, A_inh, sigma_inh)

v_x, v_y = 0.0, 0.0

for t in range(n_steps):

    r_N = np.maximum(0, s_N)
```

```

r_S = np.maximum(0, s_S)
r_E = np.maximum(0, s_E)
r_W = np.maximum(0, s_W)

Irec = convolve(r_N, W_N, mode='constant') + convolve(r_S, W_S,

B_N = B0 * (1 + alpha * v_y)
B_S = B0 * (1 - alpha * v_y)
B_E = B0 * (1 + alpha * v_x)
B_W = B0 * (1 - alpha * v_x)

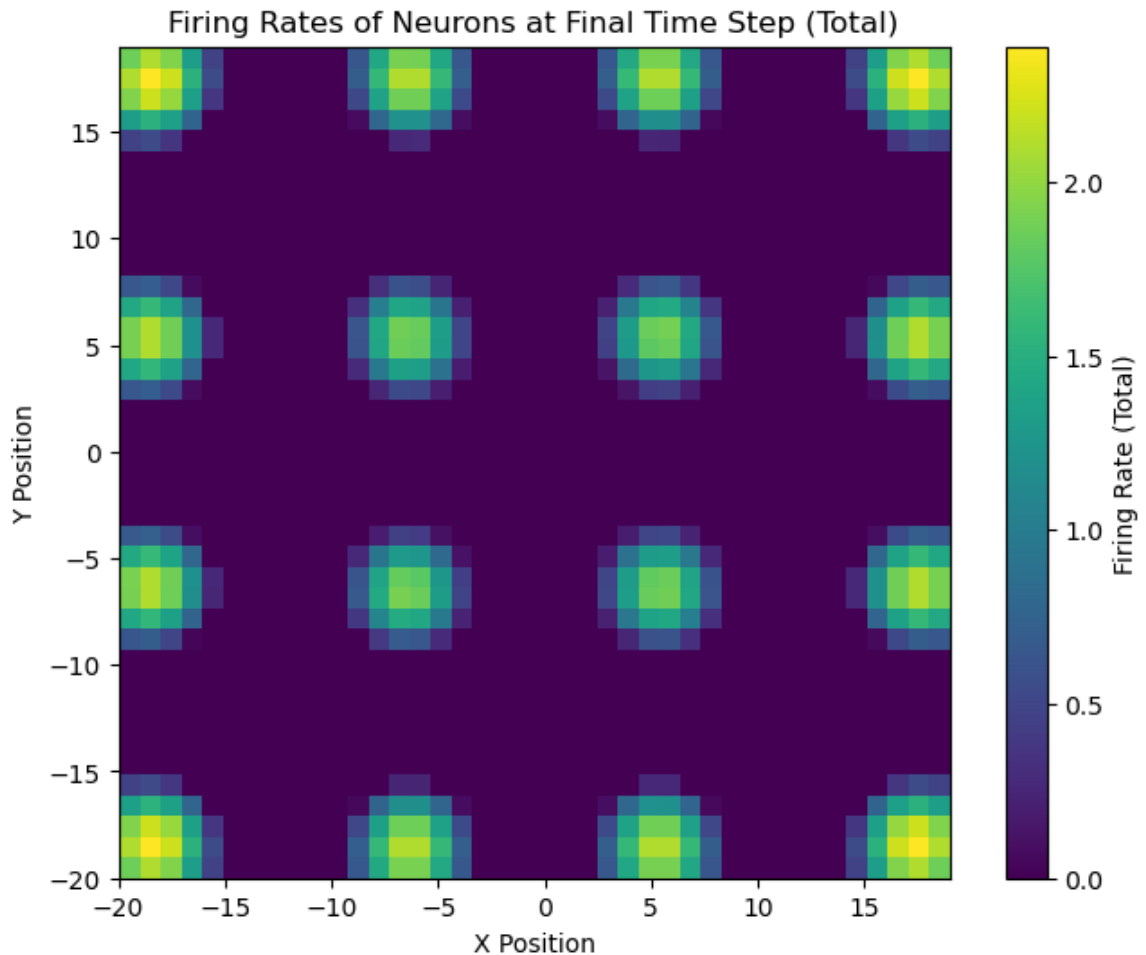
s_N += dt/tau * (-s_N + Irec + B_N)
s_S += dt/tau * (-s_S + Irec + B_S)
s_E += dt/tau * (-s_E + Irec + B_E)
s_W += dt/tau * (-s_W + Irec + B_W)

r_N_history[t] = r_N
r_S_history[t] = r_S
r_E_history[t] = r_E
r_W_history[t] = r_W

r_history_tot_4pop = r_N_history + r_S_history + r_E_history + r_W_

# plot 2d heatmap of total firing rates
plt.figure(figsize=(8, 6))
plt.imshow(r_history_tot_4pop[-1], extent=[x[0], x[-1], y[0], y[-1]]
plt.colorbar(label='Firing Rate (Total)')
plt.title('Firing Rates of Neurons at Final Time Step (Total)')
plt.xlabel('X Position')
plt.ylabel('Y Position')
plt.show()

```



Answer:

Since we put the mode constant for the convolve, neurons near the border are less inhibited than the one in the center. We can see that with the color of the heatmap: Neurons near the border are brighter (>2), even more near the corner (less inhibited horizontally and vertically), than the one in the center (around 1.75, same as 2.2). Furthermore, the symmetry of the population has changed since there is no lateral inhibition from the other side of the population; the lines are not alternated anymore, they are perfectly aligned. This is no more a hexagonal grid pattern.

To recover the grid pattern with aperiodic boundaries, there are two proposed methods:

- attenuate the outgoing weights for neurons near the boundary
- attenuate the input for neurons near the boundary

The intuition is that these methods reduce the influence of artifacts at the boundary.

We define a radial masking envelope $e(\vec{x})$ that equals 1 in the center and smoothly tapers to 0 near the boundaries:

$$e(\vec{x}) = \exp(-\gamma^2 \max(0, |\vec{x}| - R_{fade})^2)$$

where:

- $|\vec{x}|$ is the distance from the center
- $R_{fade} = f_{ratio} \cdot \frac{M}{2}$
- γ controls the exponential fall-off

Use:

- $\gamma = 0.08$
- $f_{ratio} = 0.2$

Because we want to attenuate the boundary effects, we enlarge the network to $M = 50$.

3.2

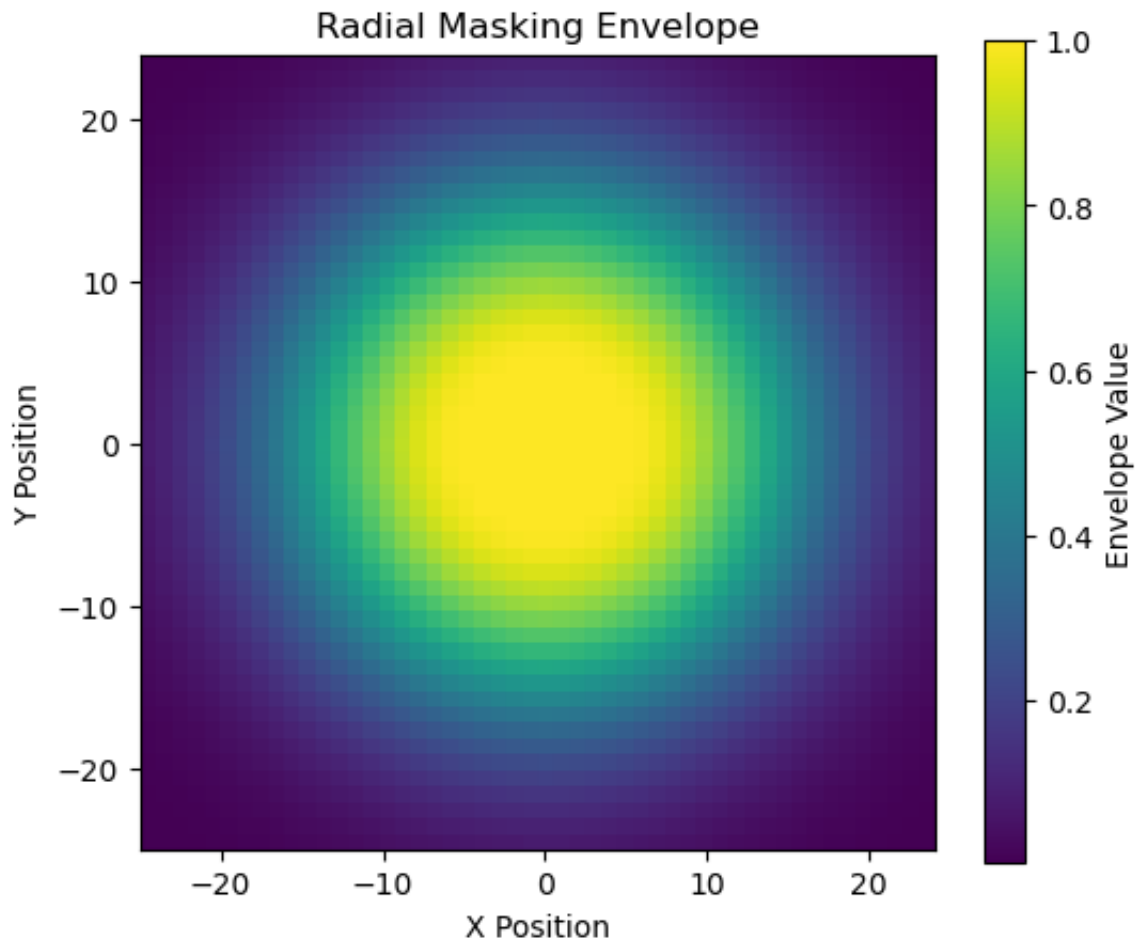
Compute and plot the 2D radial masking envelope $e(\vec{x})$.

```
In [37]: # parameters
M = 50
gamma = 0.08
f_ratio = 0.2
R_fade = f_ratio * M/2

# grid
x = np.linspace(-M/2, M/2, M, endpoint=False)
y = np.linspace(-M/2, M/2, M, endpoint=False)
X, Y = np.meshgrid(x, y)

e = np.exp(-gamma**2*np.maximum(0, np.sqrt(X**2 + Y**2) - R_fade)**2)

# plot radial masking envelope
plt.figure(figsize=(6, 5))
plt.imshow(e, extent=[x[0], x[-1], y[0], y[-1]], origin='lower')
plt.colorbar(label='Envelope Value')
plt.title('Radial Masking Envelope')
plt.xlabel('X Position')
plt.ylabel('Y Position')
plt.show()
```



3.3

Modify your network such that the outgoing weights of boundary neurons are scaled by $e(\vec{x})$.

(Hint: this is equivalent to scaling the firing rates $r_{N,S,E,W}$ by $e(\vec{x})$ before performing the convolution.)

Simulate with $\vec{v} = (0, 0)$ and plot the final network activity:

$$r_{total}(x, y)$$

Describe the pattern.

```
In [38]: # parameters
M = 50
gamma = 0.08
f_ratio = 0.2
R_fade = f_ratio * M/2

# grid
x = np.linspace(-M/2, M/2, M, endpoint=False)
y = np.linspace(-M/2, M/2, M, endpoint=False)
X, Y = np.meshgrid(x, y)

e = np.exp(-gamma**2*np.maximum(0, np.sqrt(X**2 + Y**2) - R_fade)**2)
```

```

A_exc = 1.0
A_inh = 1.0
sigma_exc = 4.8
sigma_inh = 5.0

tau = 10.0 # ms
dt = 1.0 # ms
l = 1.0
alpha = 0.1
B0 = 1.0

n_steps = 400

# initial potentials
rng = np.random.default_rng(seed=42)
s_N = rng.uniform(0, 0.1, size=(M, M))
s_S = rng.uniform(0, 0.1, size=(M, M))
s_E = rng.uniform(0, 0.1, size=(M, M))
s_W = rng.uniform(0, 0.1, size=(M, M))

r_N_history = np.zeros((n_steps, M, M))
r_S_history = np.zeros((n_steps, M, M))
r_E_history = np.zeros((n_steps, M, M))
r_W_history = np.zeros((n_steps, M, M))
r_history_tot_4pop = np.zeros((n_steps, M, M))

W_N = mexican_hat_2d(X, Y-l, A_exc, sigma_exc, A_inh, sigma_inh)
W_S = mexican_hat_2d(X, Y+l, A_exc, sigma_exc, A_inh, sigma_inh)
W_E = mexican_hat_2d(X-l, Y, A_exc, sigma_exc, A_inh, sigma_inh)
W_W = mexican_hat_2d(X+l, Y, A_exc, sigma_exc, A_inh, sigma_inh)

v_x, v_y = 0.0, 0.0

for t in range(n_steps):

    # scaling with envelope
    r_N = np.maximum(0, s_N) * e
    r_S = np.maximum(0, s_S) * e
    r_E = np.maximum(0, s_E) * e
    r_W = np.maximum(0, s_W) * e

    Irec = convolve(r_N, W_N, mode='constant') + convolve(r_S, W_S,

    B_N = B0 * (1 + alpha * v_y)
    B_S = B0 * (1 - alpha * v_y)
    B_E = B0 * (1 + alpha * v_x)
    B_W = B0 * (1 - alpha * v_x)

    s_N += dt/tau * (-s_N + Irec + B_N)
    s_S += dt/tau * (-s_S + Irec + B_S)
    s_E += dt/tau * (-s_E + Irec + B_E)
    s_W += dt/tau * (-s_W + Irec + B_W)

    r_N_history[t] = r_N
    r_S_history[t] = r_S

```

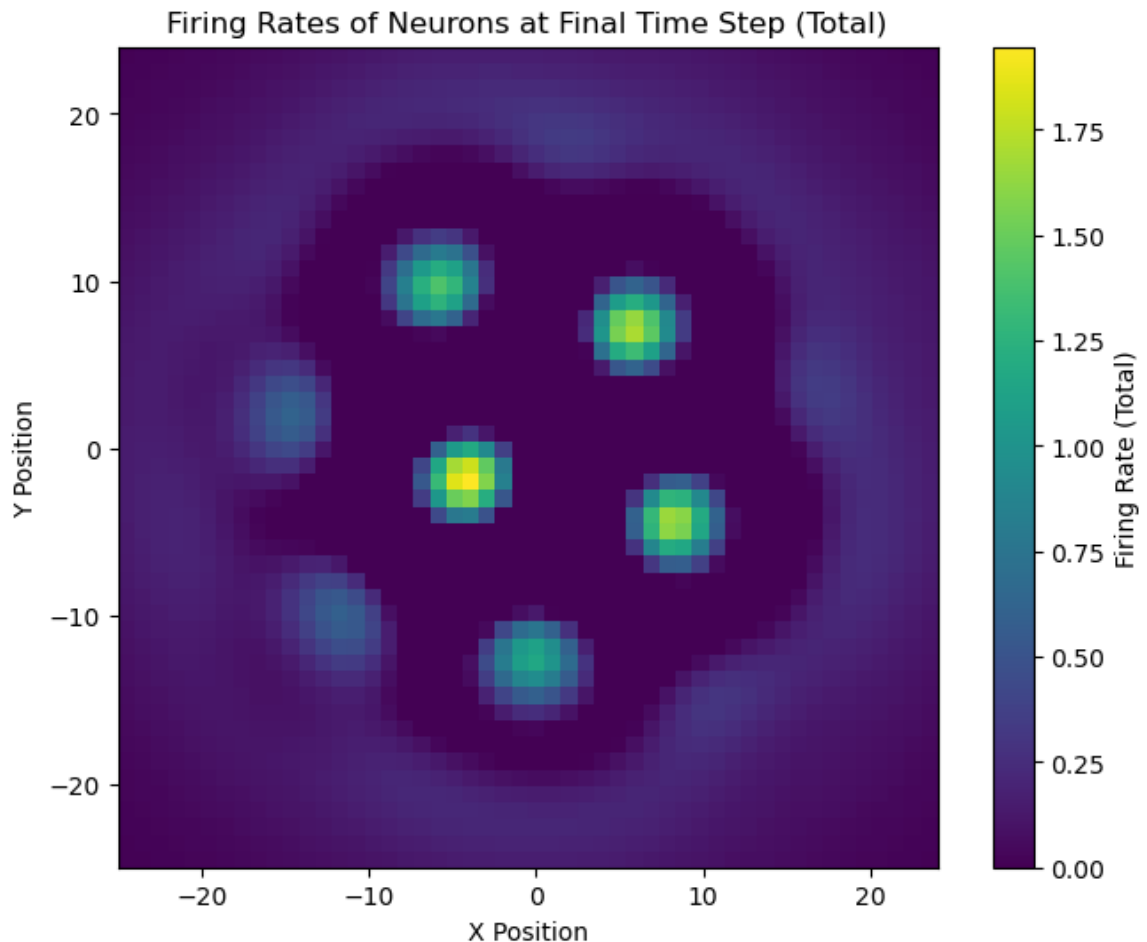
```

r_E_history[t] = r_E
r_W_history[t] = r_W

r_history_tot_4pop = r_N_history + r_S_history + r_E_history + r_W_

# plot 2d heatmap of total firing rates
plt.figure(figsize=(8, 6))
plt.imshow(r_history_tot_4pop[-1], extent=[x[0], x[-1], y[0], y[-1]]
plt.colorbar(label='Firing Rate (Total)')
plt.title('Firing Rates of Neurons at Final Time Step (Total)')
plt.xlabel('X Position')
plt.ylabel('Y Position')
plt.show()

```



Answer:

With the envelope scaling the firing rate before the convolution, boundary neurons contribute less to the recurrent input, therefore the boundary artifact of 3.1 is attenuated. Now we see a higher activity in the center since $e = 1$ there and the further we are from the center, the weaker neurons activity become. The pattern is not perfectly regular since e breaks the translational symmetry of the network because e scales the outgoing contribution of the neurons on the border to 0 so the lateral inhibition changes in the system, that is why we can see a different grid pattern.

3.4

Instead of attenuating the weights, modulate the velocity inputs:

Multiply B_N, B_S, B_E, B_W by the envelope $e(\vec{x})$ (making the input position-dependent).

Repeat the simulation and plot as in Ex. 3.3.

Comment on the resulting pattern.

```
In [39]: # parameters
M = 50
gamma = 0.08
f_ratio = 0.2
R_fade = f_ratio * M/2

# grid
x = np.linspace(-M/2, M/2, M, endpoint=False)
y = np.linspace(-M/2, M/2, M, endpoint=False)
X, Y = np.meshgrid(x, y)

e = np.exp(-gamma**2*np.maximum(0, np.sqrt(X**2 + Y**2) - R_fade)**2)

A_exc = 1.0
A_inh = 1.0
sigma_exc = 4.8
sigma_inh = 5.0

tau = 10.0 # ms
dt = 1.0 # ms
l = 1.0
alpha = 0.1
B0 = 1.0

n_steps = 400

# initial potentials
rng = np.random.default_rng(seed=42)
s_N = rng.uniform(0, 0.1, size=(M, M))
s_S = rng.uniform(0, 0.1, size=(M, M))
s_E = rng.uniform(0, 0.1, size=(M, M))
s_W = rng.uniform(0, 0.1, size=(M, M))

r_N_history = np.zeros((n_steps, M, M))
r_S_history = np.zeros((n_steps, M, M))
r_E_history = np.zeros((n_steps, M, M))
r_W_history = np.zeros((n_steps, M, M))
r_history_tot_4pop = np.zeros((n_steps, M, M))

W_N = mexican_hat_2d(X, Y-l, A_exc, sigma_exc, A_inh, sigma_inh)
W_S = mexican_hat_2d(X, Y+l, A_exc, sigma_exc, A_inh, sigma_inh)
W_E = mexican_hat_2d(X-l, Y, A_exc, sigma_exc, A_inh, sigma_inh)
W_W = mexican_hat_2d(X+l, Y, A_exc, sigma_exc, A_inh, sigma_inh)
```

```

v_x, v_y = 0.0, 0.0

for t in range(n_steps):

    r_N = np.maximum(0, s_N)
    r_S = np.maximum(0, s_S)
    r_E = np.maximum(0, s_E)
    r_W = np.maximum(0, s_W)

    Irec = convolve(r_N, W_N, mode='constant') + convolve(r_S, W_S,

# scaling with envelope
    B_N = B0 * (1 + alpha * v_y) * e
    B_S = B0 * (1 - alpha * v_y) * e
    B_E = B0 * (1 + alpha * v_x) * e
    B_W = B0 * (1 - alpha * v_x) * e

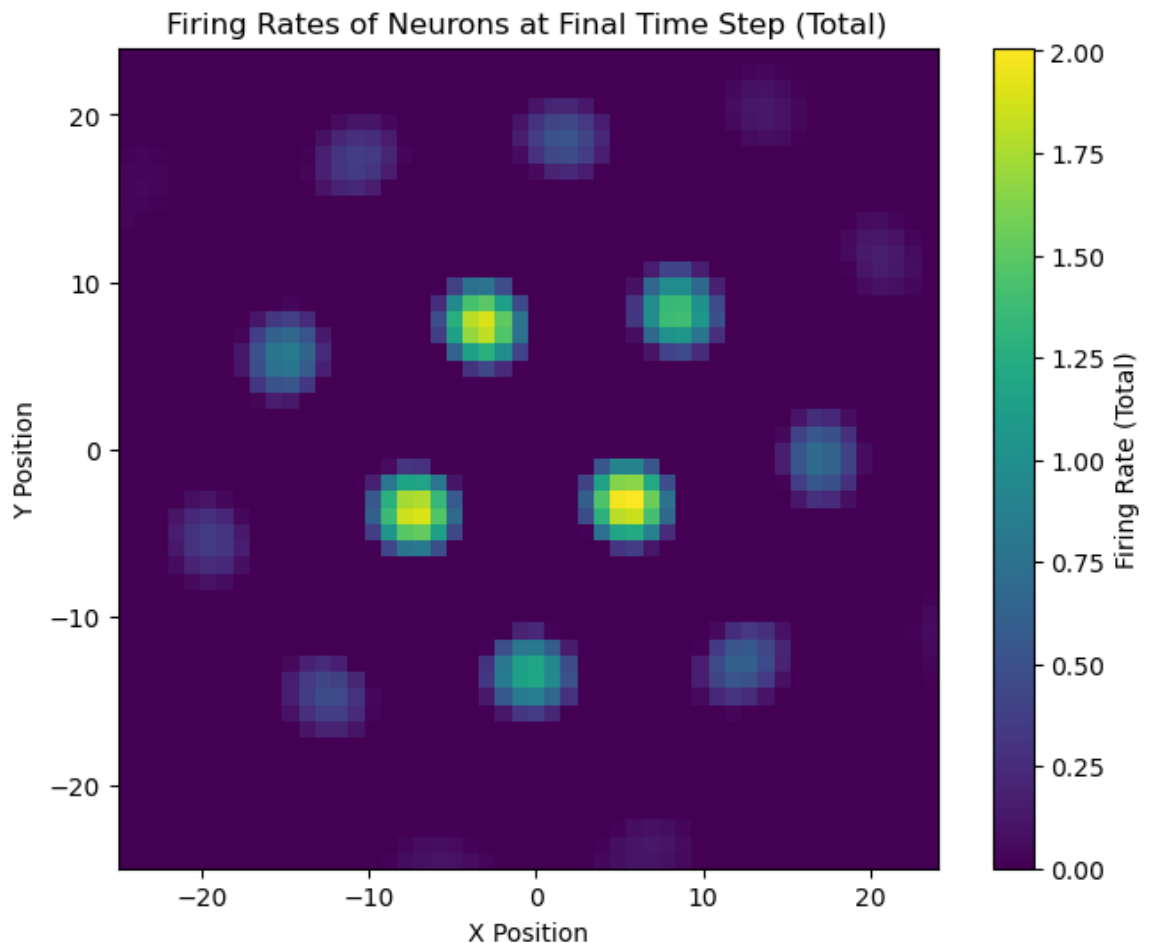
    s_N += dt/tau * (-s_N + Irec + B_N)
    s_S += dt/tau * (-s_S + Irec + B_S)
    s_E += dt/tau * (-s_E + Irec + B_E)
    s_W += dt/tau * (-s_W + Irec + B_W)

    r_N_history[t] = r_N
    r_S_history[t] = r_S
    r_E_history[t] = r_E
    r_W_history[t] = r_W

r_history_tot_4pop = r_N_history + r_S_history + r_E_history + r_W_

# plot 2d heatmap of total firing rates
plt.figure(figsize=(8, 6))
plt.imshow(r_history_tot_4pop[-1], extent=[x[0], x[-1], y[0], y[-1]]
plt.colorbar(label='Firing Rate (Total)')
plt.title('Firing Rates of Neurons at Final Time Step (Total)')
plt.xlabel('X Position')
plt.ylabel('Y Position')
plt.show()

```



Answer:

Instead of scaling the outgoing contribution, e scales the feedforward input B of boundary neurons. Neurons near the boundaries receive less drive and therefore fire less. Unlike in 3.3, where boundary neurons could still be active (they received full B) but their outgoing influence was suppressed, here their actual firing rate is reduced. However, since their r is not scaled before the convolution, they still contribute to the recurrent input of their neighbors.

3.5

What is the difference between the two approaches in Ex. 3.3 and 3.4?

Is one better than the other?

Does this have biological consequences or implications?

Answer:

In 3.3 we scale the outgoing contribution (we control what they send to other neurons) and in 3.4 we scale the input (we control what they receive from the other neurons).

Therefore 3.4 is more biologically plausible since the attenuation comes from a reduced external drive that could naturally arise from a topographic projection of another brain area. But in 3.3, each neuron would need to precisely know its position to scale its own synaptic output.

3.6 (Bonus)

Use the model in Ex. 3.4.

Generate a velocity signal with fixed amplitude and step-wise changing direction over time.

Estimate the velocity from the drifting of the grid patterns.

Does it track well the input velocity?

Do you notice any change in the pattern other than following the velocity input?

```
In [40]: # 3.6 – Velocity estimation with aperiodic boundaries (model 3.4)
# e, x, y, X, Y, W_N/S/E/W, M=50 déjà définis dans 3.4

speed = 1.0
n_steps = 700 # 300ms stabilisation + 4x100ms vitesse

# Signal : amplitude fixe, direction change tous les 100ms (droite,
angles = [0, np.pi/2, np.pi, 3*np.pi/2]
v_x_signal = np.zeros(n_steps)
v_y_signal = np.zeros(n_steps)
for i, angle in enumerate(angles):
    v_x_signal[300 + i*100 : 300 + (i+1)*100] = speed * np.cos(angle)
    v_y_signal[300 + i*100 : 300 + (i+1)*100] = speed * np.sin(angle)

# Initialisation
rng36 = np.random.default_rng(seed=42)
s_N = rng36.uniform(0, 0.1, size=(M, M))
s_S = rng36.uniform(0, 0.1, size=(M, M))
s_E = rng36.uniform(0, 0.1, size=(M, M))
s_W = rng36.uniform(0, 0.1, size=(M, M))
r_history_tot_36 = np.zeros((n_steps, M, M))

# Simulation (identique à 3.4)
for t in range(n_steps):
    v_x = v_x_signal[t]
    v_y = v_y_signal[t]

    r_N = np.maximum(0, s_N)
    r_S = np.maximum(0, s_S)
    r_E = np.maximum(0, s_E)
    r_W = np.maximum(0, s_W)

    Irec = (convolve(r_N, W_N, mode='constant') + convolve(r_S, W_S
```

```

        convolve(r_E, W_E, mode='constant') + convolve(r_W, W_W

B_N = B0 * (1 + alpha * v_y) * e
B_S = B0 * (1 - alpha * v_y) * e
B_E = B0 * (1 + alpha * v_x) * e
B_W = B0 * (1 - alpha * v_x) * e

s_N += dt/tau * (-s_N + Irec + B_N)
s_S += dt/tau * (-s_S + Irec + B_S)
s_E += dt/tau * (-s_E + Irec + B_E)
s_W += dt/tau * (-s_W + Irec + B_W)

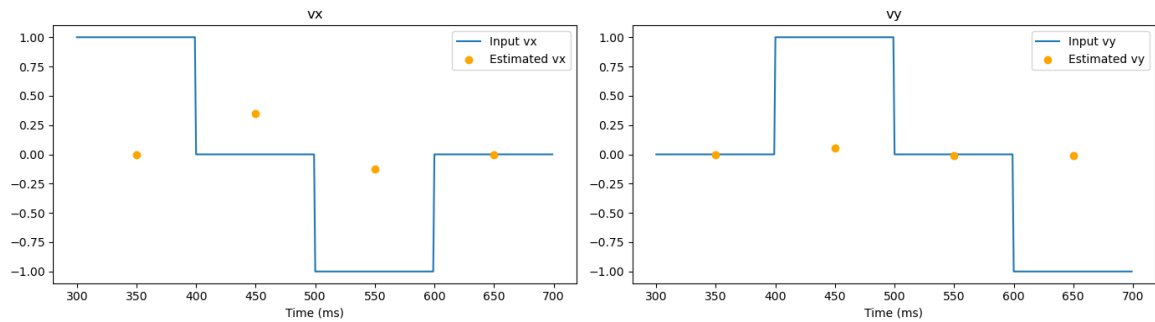
r_history_tot_36[t] = r_N + r_S + r_E + r_W

# Track bump (argmax, identique à 2.4)
bump_x = np.zeros(400)
bump_y = np.zeros(400)
for t in range(300, 700):
    idx = np.unravel_index(np.argmax(r_history_tot_36[t]), (M, M))
    bump_y[t-300] = y[idx[0]]
    bump_x[t-300] = x[idx[1]]

# Estimation au milieu de chaque segment de 100ms
est_times = [350, 450, 550, 650]
est_vx, est_vy = [], []
win = 50
t_win = np.arange(win)
for t_est in est_times:
    i_end = t_est - 300
    i_start = i_end - win
    seg_x = np.unwrap(bump_x[i_start:i_end], period=M)
    seg_y = np.unwrap(bump_y[i_start:i_end], period=M)
    slope_x, _ = np.polyfit(t_win, seg_x, 1)
    slope_y, _ = np.polyfit(t_win, seg_y, 1)
    est_vx.append(slope_x)
    est_vy.append(slope_y)

# Plot
t_axis = np.arange(300, 700)
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
axes[0].plot(t_axis, v_x_signal[300:], label='Input vx')
axes[0].scatter(est_times, est_vx, color='orange', zorder=5, label=
axes[0].legend(); axes[0].set_title('vx'); axes[0].set_xlabel('Time
axes[1].plot(t_axis, v_y_signal[300:], label='Input vy')
axes[1].scatter(est_times, est_vy, color='orange', zorder=5, label=
axes[1].legend(); axes[1].set_title('vy'); axes[1].set_xlabel('Time
plt.tight_layout()
plt.show()

```



Exercise 4 — Final Question

Please select **ONLY ONE** of the following three questions and answer it in half a page of free text.

Conceptually this is the most challenging part of the miniproject! You need to think deeply and really understand the issues before you answer.

You can choose the question that fits best your background and interests.

For this final question you are **NOT** allowed to discuss with other teams of students working on the same project.

4.1 (Theory-oriented)

Assuming that your project works well and has all the qualitative features that you expect it to have.

Can you think of a special case of the model for which you can derive mathematically the exact answer, using the mathematical theories covered in class (for example field models)?

How could you use this to **PRECISELY** test your program, to make sure that it is quantitatively correct?

Hint: What type of mismatch would you expect if either the theory calculations or the model simulations are wrong by a factor of 2 at an unknown location?

Test your model for quantitative correctness.

4.2 (Coding-oriented)

This miniproject was inspired by the paper from Burak and Fiete (2009).

In the paper, they described their simulation and parameters in the following rephrased paragraph (on the next page).

What is the EXACT mapping between your code and the description in the paper?

If there are differences, what are they?

4.3 (Biology-oriented)

This miniproject in computational neuroscience should have a link to neurobiology.

Be creative and design an experiment that would enable you to test specific aspects of the model predictions.

If necessary, think of additional modules or features that you would have to add to your model to make a comparison with experiments possible.